DEEP REINFORCEMENT LEARNING FOR CHUNKS RETRIEVAL

A thesis presented at OPIT - Open Institute of Technology

in partial fulfillment of the requirements for the degree of

Master of Science (MSc) in Responsible AI

by

Fabio Tessarollo

St. Julian's, Malta

April, 2026

Approval of the Thesis

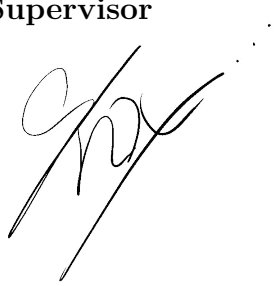
DEEP REINFORCEMENT LEARNING FOR CHUNKS RETRIEVAL

This thesis by Fabio Tessarollo has been approved by faculty below, who recommend it be accepted in partial fulfillment of requirements for the degree of Master of Science in Responsible AI.

Thesis Supervisor

Sina Famouri

14 March 2026



Thesis Defense Examining Committee

Abhinay Pandya

Prithviraj Dasgupta

Abstract

In Retrieval-Augmented Generation (RAG) systems, documents are typically divided into fixed-size chunks, which ignore the semantic structure of text. This causes fragmentation of information, complicating the retrieval of chunks that capture only a small portion of the relevant passage. Strategies such as hierarchical, overlapping or semantic chunking are the prevailing approaches, but they may introduce noise, increase the number of tokens to process, or significantly raise computational costs.

As an alternative, we propose a Deep Reinforcement Learning model that sequentially navigates a similarity-based rank of fixed-size chunks and dynamically explores relationships between adjacent chunks. The model operates on top of pre-trained, frozen text embeddings, whose semantic representations are processed to enable a refined selection of chunks, improving upon a baseline solely relying on similarity scores.

To evaluate the proposed retrieval strategy, we construct a new benchmark derived from the TREC-CAR dataset and compare our model against a cosine similarity threshold baseline using the same frozen embeddings. Experimental results show a 12.8% improvement in F1 score and a 11% increase in recall, without degrading precision.

These results point toward reinforcement learning as a viable enhancement to the standard retrieval stage in RAG pipelines, requiring no additional text to be embedded beyond the fixed-size chunks, and no Large Language Model (LLM) to be fine-tuned.

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Sina Famouri, for his guidance, support, and encouragement throughout this research.

Contents

List of Tables

List of Figures

Chapter 1

Introduction

Nowadays, information retrieval (IR) of text is increasingly often leveraging language models (LM), in contrast to sparse techniques. This approach adopts embedding models to generate multidimensional dense representations of text (e.g., Wang et al., 2022), used to compare the query with text chunks candidates, obtained by splitting a larger text corpus that is expected to contain the answer to the query. This vector comparison allows for the computation of a similarity score between the chunk representation and the query representation, resulting in a relevance metric useful to determine whether the text portion should be considered or not to answer the query, and therefore retrieved.

Normally, chunks are ranked according to the cosine similarity, a metric based on the angular difference between the vectors. The candidates are ranked with this metric, and the first top-k are extracted (e.g., Karpukhin et al., 2020). This retrieval technique can find its application in Retrieval-Augmented Generation (RAG) applications, where the extracted content is attached to the user prompts to support Large Language Models (LLM) chat-bots in the process of generating an informed answer.

The quality of the answer is clearly based on the success of the retrieval, the task on which this work is focused on. Thanks to a diverse set of supervised datasets, it is possible to evaluate this retrieval quality and compare different embedding models or retrieval techniques (e.g., Kwiatkowski et al., 2019; Yang et al., 2018; Nanni et al., 2017).

For this evaluation, the preferred metric is usually recall, measuring the quota of relevant information recovered, independently of extracted false positives. This is due to the

ability of modern LLMs to exclude secondary information that is not useful to answer. Therefore, these retrieval models are usually optimized for not losing any relevant text portion rather than not retrieving irrelevant ones. While this approach can be successful, it still introduces noise and costly extra tokens that the LLMs must process.

At the same time, the text portions available to retrieve are usually fixed in size, causing relevant paragraphs to be truncated at arbitrary points, which can fragment the narrative flow and lead to partial or incomplete contextual information. The smaller truncated portion, even if shorter compared to the irrelevant information of the chunks it belongs to, should still be retrieved and could be fundamental to properly answer the query.

While extending the rank or using overlapping, hierarchical chunks could help in not missing these smaller pieces of information, it also introduces more irrelevant information and computation expenses. On the other hand, a shorter rank of chunks could limit the noise, but it would certainly increase the number of false negatives, especially the ones having smaller relevant information due to the fixed chunking, or simply when the relevant passage is very short if compared to the full chunk size.

Therefore, substantial room for improvement remains. In this work, we aim to address these limitations by overcoming both the inherent drawbacks of the conventional combination of fixed-size chunking and similarity-based retrieval. For a more precise extraction, the reference metric predominantly monitored for this retrieval project is F1 score, as it jointly accounts for both precision and recall (other text retrieval works that uses this metric are, e.g., Gomez-Cabello et al., 2025, Do Amaral et al., 2025). Starting from the baseline F1 score, in this work we achieve an improvement by increasing recall while maintaining the same level of precision.

The developed Reinforcement Learning (RL) model leverages the cosine similarity-based retrieval to both produce the rank of chunks, the starting point of the model environment,

and as baseline for the evaluation. The developed model ultimately improves the retrieval quality according to a dedicated new benchmark built from the open source TREC-CAR dataset.

More specifically, Double and Dueling Deep Q-Networks (DDDQN) are employed to sequentially explore the cosine similarity rank of chunks for a given query and decide whether to skip or take single chunks or the same chunks merged with their adjacent chunks from the original text corpus. This evaluation takes place while considering a representation of the already taken text information.

Normally, RL use cases are video games or physical simulations, where the data is the outcome of the rules that dominates these potentially unlimited environments and there are no overfitting prevention needs. In this project instead, we use a limited supervised dataset and leverage the cross-validation approach to ensure the model's ability to generalize.

Chapter 2

Literature Review

RAG models were first introduced jointly with LM in an end-to-end architecture (Lewis et al., 2021), combining parametric memory (learned during training) with nonparametric memory (retrieval-based) to address limitations of purely parametric LM, such as static knowledge, lack of interpretability, and hallucinations. While modern LLMs like GPT-4 (OpenAI et al., 2024), have significantly scaled parametric memory, storing vast amounts of information in their weights, they remain constrained by their training data cutoff and inability to dynamically incorporate new or private knowledge without costly fine-tuning (Bommasani et al., 2023). This limitation is particularly critical in enterprise or domain-specific settings, where chat-bots must leverage proprietary data, internal documents, or real-time updates to provide accurate, context-aware responses.

RAG systems address this gap by decoupling knowledge storage from the LM itself. Instead of relying solely on parametric memory, RAG pipelines retrieve relevant documents from an external corpus (non-parametric memory) and condition the LM on this context during inference. The retrieval process typically involves dense vector similarity search: the user's prompt is encoded into an embedding space and the top-K most semantically similar documents are fetched from a text corpora similarly encoded, in a process called "Dense Passage Retrieval" (DPR) by Karpukhin et al. (2020). These retrieved documents are concatenated with the prompt as context, enabling the LM to generate informed, domain-specific responses (Shuster et al., 2021). Unlike how it was originally conceived, the retrieval is not trained jointly with the LM, it is rather acting as an external

process, supporting the LLM by integrating user prompts with the information to answer. This decoupled paradigm is precisely the setting where our work operates in: rather than retraining any part of the pipeline, we focus on improving what the retrieval stage selects, given fixed embeddings and fixed chunks. It is against this decoupled, frozen paradigm that subsequent work has largely pushed, by fine-tuning text-reader components or re-designing chunking strategies, thus trading simplicity and generalizability for retrieval gains.

2.1 Retrieval

One key direction to improve RAG systems has been the introduction of a re-ranking stage to refine the order of the retrieved passages before they are fed to the LLM. A representative early approach is RECONSIDER, a span-focused cross-attention re-ranker that fine-tunes a pretrained reader model to better discriminate among top candidate passages (Iyer et al., 2020). By training on high-confidence predictions of a QA model and focusing on answer spans, RECONSIDER learns to eliminate false positives and significantly boosts accuracy (e.g. raising Exact Match on Natural Questions to 45.5%). This was proof that a dedicated cross-encoder can substantially improve retrieval quality by re-ordering the initial DPR results.

Another recent trend is to harness large language models directly as re-rankers by using them to evaluate the relevance of each candidate. Researchers have prompted LLMs to score passages by the likelihood of reproducing the question given that passage as context. This zero-shot LLM-based re-ranking can improve answer recall, but its effectiveness depends heavily on prompt design, and it incurs substantial computational cost if the LLM itself must be fine-tuned. This is addressed with Passage-Specific Prompt Tuning (PSPT), a parameter-efficient method that attaches learned soft prompts to each passage

instead of altering the LLM's weights. Their approach allows a frozen Llama-2 model to re-rank passages more reliably by incorporating passage-specific cues, yielding better open-domain QA performance without the expense of full model training (Wu et al., 2024).

Beyond improving how passages are ranked, research has also looked at enriching the context given to the language model. One strategy is integrating structured knowledge to complement text passages. For instance, Ju et al. develop Grape, a knowledge-enhanced reader that injects relational information from knowledge graphs into the QA process (2022). Given a question and its retrieved passages, Grape constructs a localized graph of entities and uses a graph neural network to fuse this with the reader's latent representations. This graph-informed reading mechanism improved answer exact-match scores by roughly 2 points on open-domain QA benchmarks without changing the retrieval component. Such results suggest that incorporating knowledge graph context can help the model resolve complex relationships or disambiguate entities in ways pure text might not.

This work, rather than fine-tuning a language model or incorporating an LLM as a reasoning component within the retrieval stage, trains a deep reinforcement learning model from scratch, operating solely on frozen embeddings. While the learned dynamics may be dataset-dependent, this approach avoids the computational costs that come with large pre-trained models.

One example of a re-ranking strategy operating in a setting similar to the one proposed in this work is Hybrid and Collaborative Passage Re-ranking (HybRank). Rather than scoring each passage in isolation, HybRank leverages inter-passage similarity signals by representing each passage and query as a sequence of similarity scores computed against a set of anchor passages, drawing from both sparse and dense retrievers (Zhang et al., 2023).

These hybrid sequences are then processed by two small Transformer encoders, trained from scratch, with no large pre-trained language model involved. As a result, HybRank acts as a lightweight ”plug-in” re-ranker that yields consistent gains over standard dense retrieval alone.

2.2 Chunking

Another practical aspect of RAG pipelines is how documents are chunked before retrieval. Large documents must be split into chunks for embedding and search, and a natural question is whether semantic chunking (splitting text by topical coherence or discourse boundaries) yields better results than simple fixed-size chunks. Different works report better results with dynamic strategies, such as adaptive, proposition and semantic chunking (Gomez-Cabello et al., 2025; Do Amaral et al., 2025).

However, when associated with its computational costs, empirical evidence indicates that semantic chunking provides limited gains over uniform fixed-size chunks in realistic scenarios (Qu et al., 2025). Overlapping sliding-window chunks often perform equally well or better by reducing content fragmentation. More recently, hierarchical chunking has emerged as a promising alternative (Sarthi et al., 2024), combining larger and smaller segments within a unified structure. This multi-level representation has the advantage of carrying both the semantic meaning that can be inferred from the context of a larger section and a smaller section, enabling retrieval model in recognizing the full picture of multiple paragraphs and the smaller nuances contained in sentences. A competitive work that ensemble the hierarchical strategy with a variable text segmentation is HiChunk (Lu et al., 2025), which constructs multi-granular, structure-aware chunks to preserve contextual coherence while enabling retrieval at different levels of abstraction.

Our apporoach is different: we don’t treat the chunking as a variable to optimize but

rather as a constraint, consistently with how most deployed RAG systems nowadays operate. This keeps the embedding process simple and doesn't add extra text to embed.

2.3 Benchmarks

This diverse set of retrieval strategies is tested over different benchmarks. The Natural Questions (NQ) dataset, for example, consists of real Google queries, each typically answerable by a single Wikipedia passage (Kwiatkowski et al., 2019). It provides both a long answer (paragraph) and a short answer annotation for each question. Another is HotpotQA (Yang et al., 2018), that instead requires multi-hop reasoning: questions are constructed such that a model must retrieve and synthesize information from multiple documents (often two hops) to find the answer, possibly integrating the process with a Chain of Thought (CoT) when the adopted model is a LLM. This dataset evaluates a system's ability not just to fetch relevant facts, but to connect them across passages, a much harder retrieval challenge.

The benchmark constructed to evaluate this work is described in detail in the following chapter.

Chapter 3

Benchmark and Baseline

This chapter describes the dataset selection process, the transformations applied, and the underlying motivations guiding these choices.

3.1 Requirements

To rigorously evaluate the proposed solution and ensure a fair comparison with similarity-based retrieval methods, the supervised benchmark dataset was required to provide, for each query, both the annotated relevant passages and the original text corpus from which those passages were extracted.

In fact, many datasets provide a collection of candidate chunks without the original text corpus. This wasn't fit for the project, as the designed model is supposed to considerate beyond single chunks, also their version merged with previous and next adjacent chunks, to overcome the mentioned problem of truncated relevant passages due to fixed chunking.

An additional requirement concerned the statistical properties of the relevant passages, such as their average length and variance relative to the predefined chunk size. If the average passage length were significantly smaller than the chunk size, most queries would correspond to only one, or at most few, relevant chunks. Furthermore, a low variance in passage length would imply a nearly fixed number of relevant chunks per query. Such regularity could be exploited by the model and would not reflect realistic retrieval scenarios, where the number and extent of relevant passages naturally vary. Beyond text size

variance, another ideal property of the annotated passages was the presence of multiple, separated text passages for a single query, with some degree of variance in the number of passages.

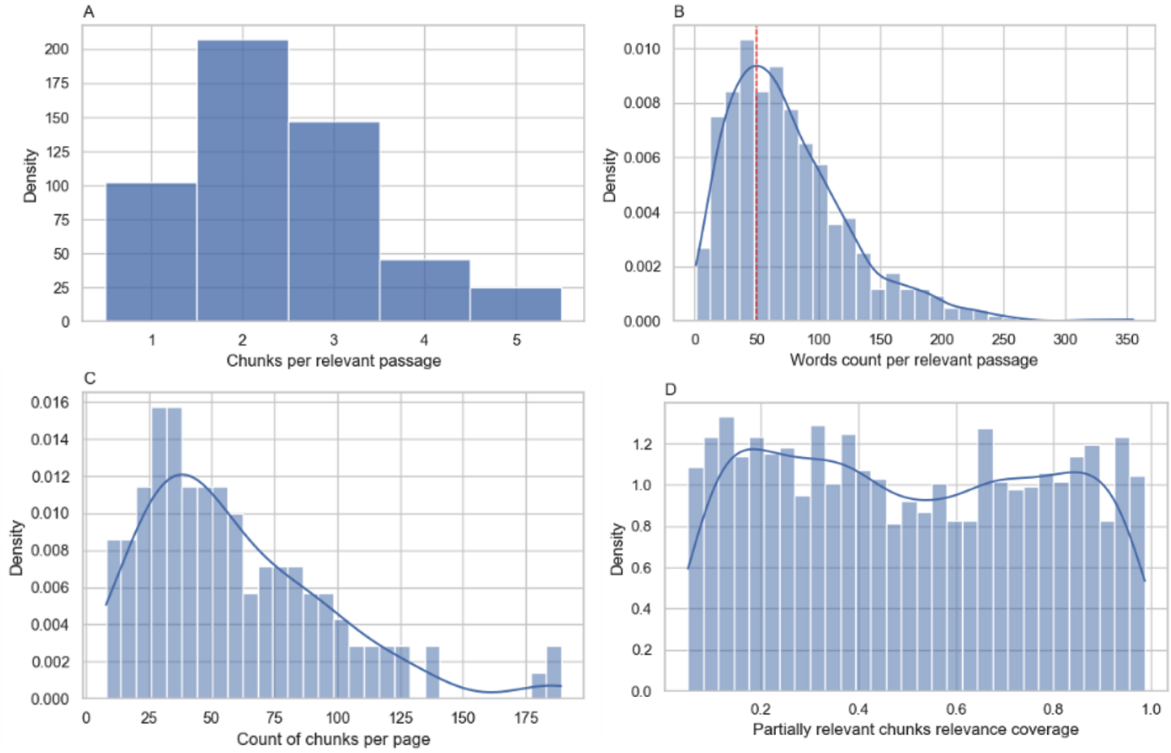
3.2 Dataset Selection

One dataset that was found, holding all of these requirements, was the TREC Complex Answer Retrieval (TREC CAR) benchmark, which adopts paragraphs headlines as queries, and relates them to annotated text portions with different lengths (that are not only found right under the headline itself) (Nanni et al., 2017). With this dataset, a single query can have multiple facets, and the task is to retrieve a set of complementary relevant passages that together cover the query requirements.

3.3 Dataset Transformation

However, TREC-CAR dataset stores entire paragraphs as candidates, rather than fixed chunks. Therefore, original pages were reconstructed by concatenating the paragraphs, and fixed chunks from the obtained pages were extracted by splitting every 50 words the pages. To label the relevant fixed chunks as relevant, for every query, the annotated relevant content was matched with the chunk content. More specifically, whenever at least 5% (at least three words, given the chunk size) of the chunk was overlapping with the relevant content, it was marked as relevant. This result was stored in a file including the query, the page identifiers and the list of relevant chunks. This approach achieved the requirement of having a sufficient variability of chunk counts related to a relevant passage, as can be evinced by the distribution of chunks per relevant passage reported in Figure 3.1A, whose average is about 2.4 chunks per passage. Similarly, we can observe

Figure 3.1.
Feature distributions from the resulting dataset. Figure B includes a red line to mark chunk size.



Note. (A) Amount of relevant chunks deriving from a single relevant passage; (B) Lengths in words count of the relevant passage, including a red line indicating the size of the fixed chunk; (C) Lengths in chunks of the Wikipedia pages; (D) Distribution of the relevance coverage of partially relevant chunks.

the distribution of lengths in words of the relevant passages, with 75 words on average, compared with the fixed size of the chunk of 50 in Figure 3.1B.

Within the obtained dataset, each query is associated with a set of relevant chunks $\mathcal{R}_q$ (with cardinality $|\mathcal{R}_q| \geq 1$) and a larger set of candidate chunks $\mathcal{C}_q$ extracted from a single Wikipedia page, where $\mathcal{R}_q \subset \mathcal{C}_q$. In the training set, on average $\mathcal{R}_q$ constitutes approximately 10% of $\mathcal{C}_q$. With this benchmark, different queries can be related to the same page, and the tested models are required to search for the relevant chunks within the single page related to the query. The relevant chunks present text possibly coming from different and distant positions of the pages, or belonging to the same long relevant portion (in this case the relevant chunks are consecutive). Along with this information,

each chunk was stored along with its quota of relevant information, which will be referred to in this work as "relevance coverage" ($C \in [0.05, 1]$). This allowed in the analysis phase to identify chunks containing smaller relevant portions, the ones targeted by this work, and further evaluate the methods specifically for these chunks. In the training set, among the partially relevant chunks (having $C < 1$), accounting for about 71% of the total chunks, the distribution of C was found to be uniform, as we can see in Figure 3.1D.

3.4 Dataset Split

The TREC-CAR dataset comes with a training set made of 535 queries regarding 117 pages and a test set made of 565 queries regarding 132 pages. In this work, the training set queries were partitioned into a new training set (60%) and a validation set (40%). The split was performed using a stratified splitting approach to ensure balanced difficulty distributions. Specifically, queries were first grouped by their source page and for each query q was assigned a score S proportional to how easy the retrieval was, based on the position in the rank (r_i) of their relevant chunks set $\mathcal{R}_q$:

$$S(q) = \frac{1}{|\mathcal{R}_q|} \sum_{i \in \mathcal{R}_q} \frac{1}{1 + r_i}$$

Pages were then stratified according to this score into three tiers (low, medium, high difficulty) and the split was applied proportionally within each tier. This ensured that both the training and validation sets contained representative samples of easy, medium, and difficult queries, preventing skewed difficulty distributions that could bias model development. All queries from the same page were kept in the same split to avoid data

leakage. The last precaution was to sort the queries by the score, to ensure a deterministic split consistent in different runs.

The training set was used to find the best similarity threshold of the cosine similarity method and for the training of the RL model. The validation set was used to compare the results of the proposed method with the baseline, and to prevent overfitting when picking the best architecture. The best identified hyperparameters and architecture were later used for the training of the final model, utilizing the entire original training set. This final model was used to make inference on the test set and for the final evaluation and analysis.

3.5 Text Embedding

To embed the queries and the obtained fixed chunks into dense vectors, in this work we leveraged the embedding model e5-base-v2 (Wang et al., 2022), which is based on a 12-layer transformer, and it has an output dimensionality of 768.

This model was optimized for information retrieval and text classification. It was developed through weakly supervised contrastive learning, which encourages semantically similar texts to have closely aligned embeddings. The resulting vectors are L2-normalized, facilitating their use in tasks that rely on cosine similarity measures, since all vectors have a one-unit length and differ only by angle.

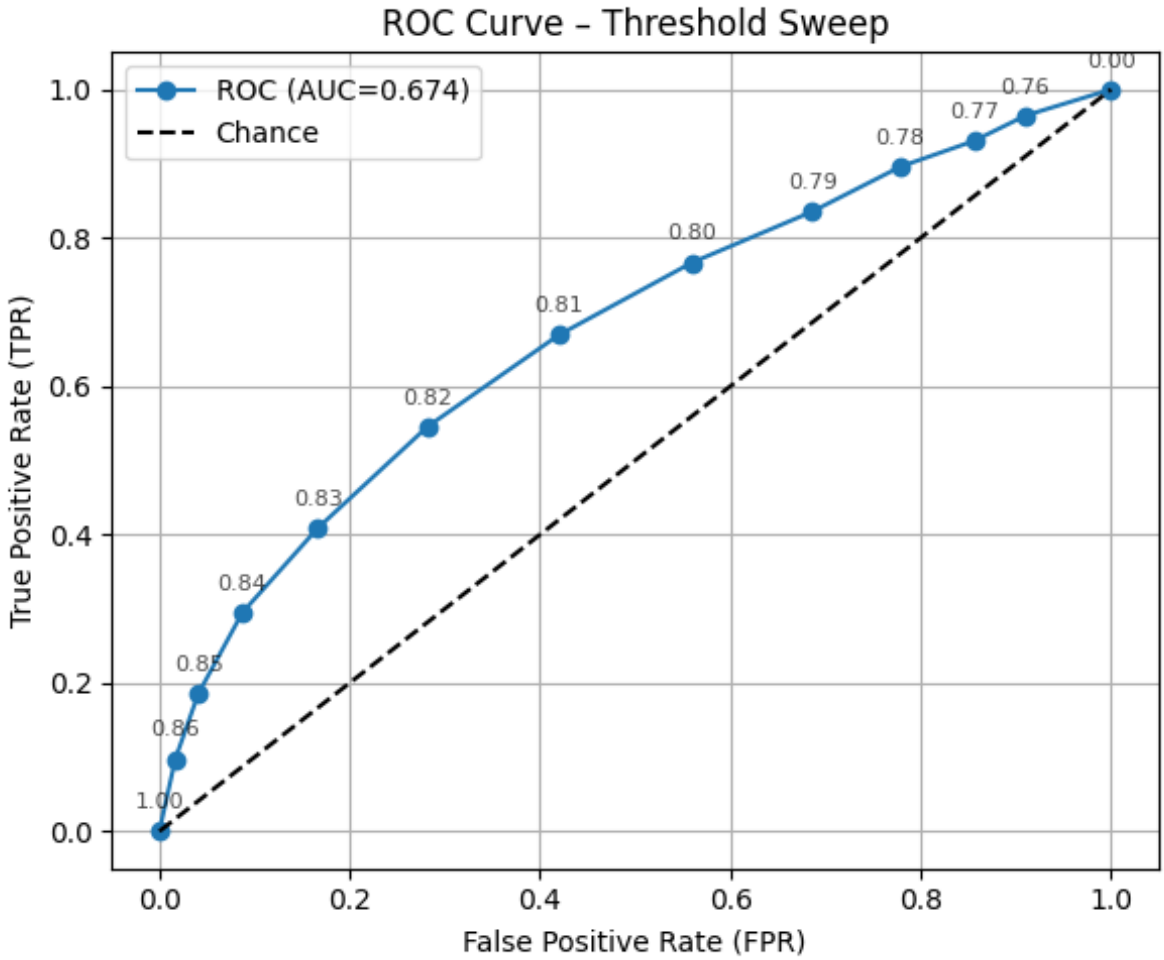
While in other solutions that are leveraging deep learning to improve the retrieval, such as RECONSIDER (Iyer et al., 2020), there is a fine tuning of the pre-trained embedding model, in this work the baseline and the RL model are evaluating chunks on the basis of the same frozen model resulting embeddings, obtained using zero-shot inference of the chosen embedding model.

3.6 Cosine Similarity Retrieval

The baseline scores were obtained by retrieving the chunks above a similarity threshold with the query, using the training set as a reference to identify the threshold. The conventional top-K extraction was here replaced with this approach to have a baseline more sensible to irrelevant information and able to retrieve a variable set of chunk candidates. The threshold was selected using the training set through a brute-force search over multiple similarity values, choosing the value that maximized the F1 score (the same similarity threshold was found with both the original full training set and the 60% split discussed in the previous chapter). ROC-AUC analysis was also employed to observe potential other threshold candidates (Figure 4.1), confirming the optimal 0.82 threshold found with brute force, achieving a F1 score of 0.2580 and a recall of 0.6621 on the validation set.

The ranks used by the RL model were generated with two fine-tuned constraining parameters: a maximum number of chunks (top-K), set to 40, and a minimum similarity, set to 0.77, which should cover for about 92%of the relevant chunks.

Figure 3.2.
ROC-AUC analysis reporting the cosine similarity thresholds in the curve labels.



Note. This plot is based on the training set split (60%), where the 0.82 threshold returned a F1 score of 0.2295 and a recall under 0.6.

Chapter 4

Proposed Solution

The intuition behind the developed solution is mirroring the process a human might follow for selecting relevant text passages to answer a given query, leveraging an initial rank based on the cosine similarity method. So, typically the agent would start from the first chunk in the rank and assess whether it is relevant. The agent might notice that the chunk is incomplete, since the fixed chunking removed a relevant last (or initial) portion of the paragraph. The agent may therefore inspect the surrounding text in the original document to recover the missing context and determine whether it should be included in the retrieved content.

While some adjacent chunks may already appear in the similarity-based ranking, others remain excluded because their similarity with the query is low. These chunks are often only partially related to the query but may contain contextual information necessary to complete a relevant passage. Recovering such partially relevant adjacent chunks (with relevance coverage $C < 1$), without hurting precision, is the objective of this work.

The agent would then proceed through the ranking, repeating the same evaluation process for subsequent chunks. This should happen while always taking into account what was already selected and how complete the selection is to answer the query.

This decision process is modeled using reinforcement learning. At a high level, each query corresponds to a learning episode. The environment state is represented through embedding-based observations and associated metadata, while a deep neural network estimates action values that guide the exploration of the ranked chunk list and the selection

of relevant passages. Learning occurs through the experience of past transitions of states, actions and rewards. The rewards are computed according to the supervised dataset documented in Chapter 3.

4.1 Environment

In this project, a custom reinforcement learning environment is implemented from scratch as a Python class, following the design principles of OpenAI Gym (Brockman et al., 2016). Each environment instance represents a single episode, corresponding to a specific topic defined by a query-page pair. The agent interacts with this environment by selecting actions, after which the environment returns the resulting states and rewards. While in most RL works the environment is a fixed set of pre-defined laws, in this work the environment was fine-tuned exactly as the model behind the learned policy. The adjustment of the architectures and variables constituting this environment, was executed with the empirical caution of ensuring a framework that can generalize with other datasets.

4.1.1 Embedding States and Action Space

The states of the environment, inputs of the trained model, are represented by the embeddings of the chunks currently examined by the agent at a given step within an episode. Actions enable the model to collect a reward based on whether the chunk is relevant and to transit to the next state.

Initial Setting. The first state space set up for the agent was composed of the embeddings:

- Query embedding (768-dimensional).
- Current chunk (or adjacent chunk) embedding (768-dimensional).

- Selected chunks, mean vector of the embeddings from the selected chunks (768-dimensional). This object will be referred in this work as ”bag of chunks”.

In this initial design, these states were paired with the following action space:

- Go to the previous chunk in the rank.
- Go to the next chunk in the rank
- Check next chunk in the page (adjacent chunk).
- Check previous chunk in the page (adjacent chunk).
- Take: include the chunk observed in the set of selected chunks, the bag of chunks.
- Submit: retrieve the created bag of chunks.

This architecture was so designed to give the agent the maximum freedom of exploration. However, the model was struggling to converge, as the set of action paths making sense was too little compared to the full set of possible paths, e.g. the adjacent chunks check actions don't work as intended when taken multiple times in a row, as well as allowing the agent to go back or forward along the rank was more resulting in redundant reversals rather than a thoughtful reconsidering of the previous decisions, as it was designed for. Even discouraging these inconvenient trajectories through the reward function didn't result in a stable behavior of the agent. Moreover, the submit action was never intentionally taken by the agent and every episode was truncated by the environment.

Final Setting. Better results were obtained with a set of actions that even with a random agent could result in a reasonable sequence of decisions. This was achieved by expanding the state information with the next and previous chunk combined with the current chunk using a projection head. In this way, the actions of checking the adjacent chunks were not needed anymore, reducing the complexity of the decision making for

the agent. Moreover, beyond the submit action, in the previous architecture the agent abilities were separated into rank exploration, achieved by going up and down in the rank and by checking adjacent chunks, and chunk picking, with the "take" action. Instead, in the final set up whichever action is taken, the exploration continues to the next chunk of the rank, reducing the agent's degrees of freedom. The submit action was removed, as the exploration was reaching its programmed bottom in every greedy episode also with this design. The final action space was defined as:

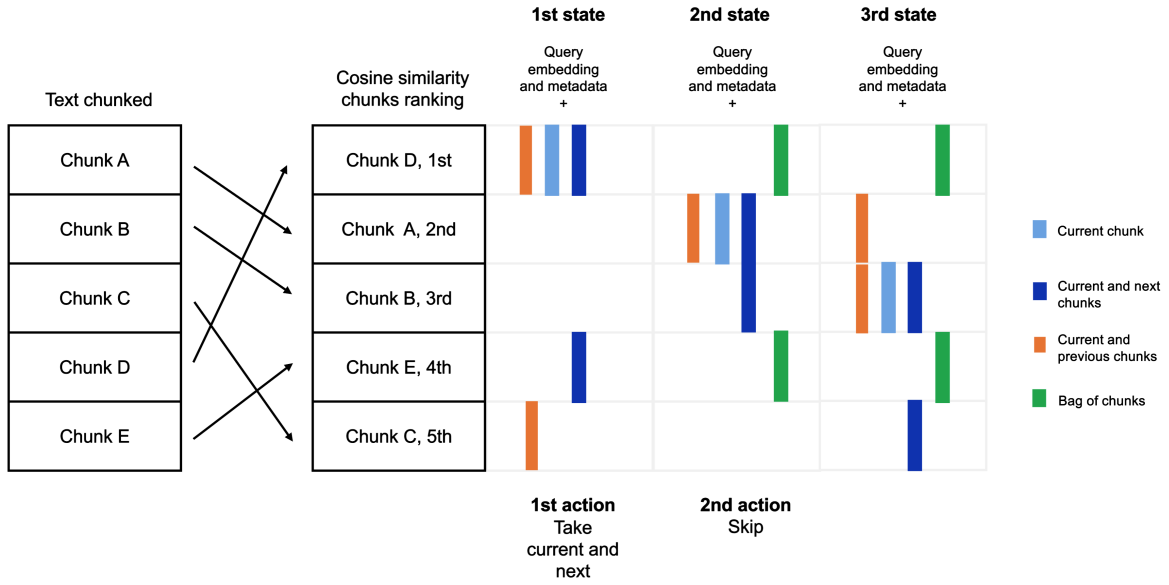
- Skip: go next in the rank, without taking the current or the adjacent chunks.
- Take Current: include the current (central) chunk in the bag of chunks.
- Take Current and Next: include the current and the next chunk.
- Take Current and Previous: include the current and the previous chunk.
- Take Triple: include the current with both adjacent chunks.

This action space was paired with a state composed of the following embeddings concatenation:

- Query embedding (768-dimensional)
- Current (central) chunk embedding (768-dimensional).
- Concatenation of current and next chunk embeddings (1536-dimensional).
- Concatenation of current and previous chunk embeddings (1536-dimensional).
- Bag of chunks, mean vector of the embeddings from the selected chunks (768-dimensional).

With this action-state design, the agent is aware at every step of the adjacent chunks. The inclusion of the bag of chunks into the state representation, should help the model

Figure 4.1.
Example of states progression.



Note. Example of state progression with references to chunk positions in the rank and in the page. In the first state, beyond the first chunk in the rank (chunk D), the agent observes its concatenation with the previous in the page (chunk C) and the next in the page (chunk E). After the "Take Current and Next" action, in the second state the current chunk becomes the second in the rank (chunk A) and we additionally find the bag of chunks updated with the chunks retrieved in the previous transition (D and E). The action that follows is a "Skip", thus in the third state the bag doesn't change, only current and adjacent chunks are updated as before.

with redundancy, coverage, and interaction between selected chunks, making the environment closer to satisfying the Markov property, according to which the transition dynamics depend only on the current state, and not on the full history of past states and actions. This feature contributes to distinguishing this approach from a classical classification task, where every chunk relevance is assessed individually, independently from the retrieval process and the past chunks. An example of state progression with the detailed environment is reported in Figure 5.1.

4.1.2 Episode Termination

With the new setting, the exploration was going in one direction only, but if it reached the bottom of the rank, it started over with a new loop. This was initially allowed to let

the agent “change his mind” given the updated bag of chunks. However, this behavior was never observed and better results were achieved without further exploration after the last chunk in the rank. Thus, while in typical reinforcement learning environments there is a timeout of the episode, truncating the agent interaction with the environment, in this work the interruption was forced externally when no new chunks were left to evaluate. Initially, this was performed by forcing the submit action and assigning a larger, terminal reward proportional to the achieved F1 score. In the final setting, as reported before, this action was removed along with its reward because it was never used by the model. But, to still keep the direct F1 score signal, this information was integrated as a component of the reward of every action.

4.1.3 Edge Cases

When the rank points to a chunk that is at the start or at the bottom of the page, the actions retrieving adjacent chunks had to be handled to avoid overflow. In a first iteration, the reward was set to zero in these cases, hoping for the agent to learn how to behave. Secondly, a more stable logic was implemented by calling the Take single action whenever an action would have provoked the overflow error. The embedding states of adjacent chunks, when not available due to rank edges, were replaced with a vector of zeros.

4.1.4 Metadata

Along with the concatenated embeddings, some metadata information was included in the state observation layer. In the development, different features were tested to support the model in improving the validation results, ranging from episode insights, regarding the position in the rank or metrics indicating the previous outcomes, to different similarity metrics between the current embedding state and the query, such as the Euclidean

distance. Initially, a set of boolean features was also included to inform the agent on whether the observed chunks were already in the bag. Later, this set was removed and the retrieved chunks were silently skipped when advancing the rank, simplifying the task for the agent and increasing its focus on the embeddings. In the final architecture, the metadata included was:

Episode Insights.

- Normalized rank position, computed as the relative position of the current chunk in the ranked list.
- Normalized bag size, computed as the ratio between the number of chunks currently in the bag and the total number of ranked chunks.

Cosine Similarities.

- Query and current chunk.
- Query and next chunk.
- Query and previous chunk.
- Query and bag of chunks.

4.1.5 Reward Shaping

The rewards guide the agent into the understanding of which action to take given a state, as its objective is to maximize its discounted future rewards. In other words, the reward shaping is about mapping the outcomes of all action state pairs, in our case leveraging the supervised dataset. In this work, the aim of this function is primarily to ensure the best chunk retrieval possible in terms of F1 score, but also to encourage convenient strategies and to balance recall and precision.

Careful calibration of the reward magnitudes had a substantial impact on both the learning dynamics and the final retrieval performance. This stage required significant experimentation, as suboptimal reward shaping often led to unintended behaviors. For instance, with some configurations the average reward per epoch increased while the F1 score simultaneously decreased, indicating that the agent was optimizing the reward signal without improving actual retrieval quality. In other cases, the agent concentrated on a narrow subset of actions or exploited weaknesses in the reward formulation, again accumulating higher rewards without producing better final chunk selections. These issues were solved by adopting a consistent reward scaling across all actions, to ensure balanced incentives and prevent systematic behaviors. This was achieved by assigning a fixed amount for every classification outcome, but incorporating some variations to encourage the right adoption of some actions or strategies.

The final reward consisted of two components. The first, accounting for 25% of the total reward, was based on the F1 score and was computed as the change in F1 induced by the action. This term ensures alignment with the overall objective while reducing reward sparsity. The second component, contributing to the remaining 75%, was based on the state-action transition. The F1-delta component alone produces an unstable signal (e.g., one more true positive weights differently based on the current retrieved set). For this reason, the majority of the reward was assigned to the state-action transition, detailed in this section.

Since true negatives don't affect F1 score, in the first version of the environment these weren't considered for reward computation. Instead, true positives, false positives and false negatives were set to return an equal reward in absolute value. Experiments showed that this approach yielded minimal F1 improvement and poor recall (below 40%). A first important improvement was achieved by doubling the reward for true positives chunks.

With this set up, a ”Take Triple” action retrieving three relevant chunks was rewarded with one unit, while a skip action missing three relevant chunks was penalized with half negative unit. From this, further fine-tuning using training and validation sets was performed, considering together with F1 score two additional objectives:

False negatives penalty for the precision-recall trade-off. Despite the first recall improvement obtained by increasing true positives reward, the model was still relying too much on the Skip action, improving F1 score only with precision. To address this, the skip of relevant chunks penalty was empirically adjusted to reach one negative unit, matching in absolute value the maximum reward from ”Take Triple” action. The reward unit was composed of:

- Half negative unit, when the current chunk was relevant.
- Additionally, up to a negative quarter if also adjacent chunks were relevant.
- Additionally, up to a negative quarter based on how early in the rank the relevant chunk was.

The adjacent chunks are less penalized, since the current (central) chunk is the main reference for the model. The rank-based component was integrated to discourage the model to skip chunks found early in the rank.
Instead, for retrieval actions selecting one or two consecutive chunks, the possible adjacent false negative chunks were not penalized, again from the need to maintain a sufficient recall. This fine-tuning was performed while taking in consideration the under-sampling of the negative examples discussed in section 4.3, also impacting on the precision and recall trade-off.

Ensuring a balanced actions frequency distribution. The supervised dataset contains cases where each action represents the optimal choice, meaning a perfect policy

would employ all of them. Ablation experiments confirmed this: every time an action was removed, retrieval quality dropped. In many experiments, the actions retrieving the current chunk together with one of the adjacent chunks (next or previous) were not taken at all or very rarely during greedy evaluation. To help the model understand when to take one of these instead of the "Take Triple", a positive component, fine-tuned to 0.15, was added to the reward in the case that the non-taken chunk, among the three exposed in the state, was irrelevant. This is the only true negative case rewarded, giving the model a stronger motivation to use these actions retrieving two consecutive chunks.

Always to ensure a balanced action taking behavior from the model, the global reward shaping was adjusted with the constraint of making higher rewards related to potential higher penalties, as can be noted from the final domains:

- Skip: $[-1, 0]$
- Take Current: $[-0.17, +0.33]$
- Take Current and Next (or Previous): $[-0.33, +0.81]$
- Take Triple: $[-0.5, +1]$

This objective was achieved together with a custom distribution for sampling actions in exploration mode, which is detailed in section 4.7.

4.2 Deep Q-Networks Architecture

The neural network serves as an estimator of Q-values, the expected cumulative rewards of taking an action given the current state and the learned policy. Its outputs allow the agent to evaluate the potential long-term benefit of its available actions and accordingly select one.

The architecture chosen for this reinforcement learning retrieval agent is the Double

Dueling Deep Q-Network, based on the Dueling architecture (Wang Z. et al., 2016) and the Double DQN architecture (Van Hasselt et al., 2016). The structure is reported in Figure 5.2.

The dueling architecture is designed to decompose Q-values into state value and action advantage components, allowing the network to separately learn how good it is to be in a state versus how much better each action is relative to others in that state.

In the double DQN architecture there are two identical Deep Q-Networks, an online network whose weights are updated after every transition and it is used to pick the actions, and a target network, whose weights are updated at a fixed rate of transitions (4000 in this work, about three times per epoch) and used to compute the expected future rewards.

4.2.1 Projections

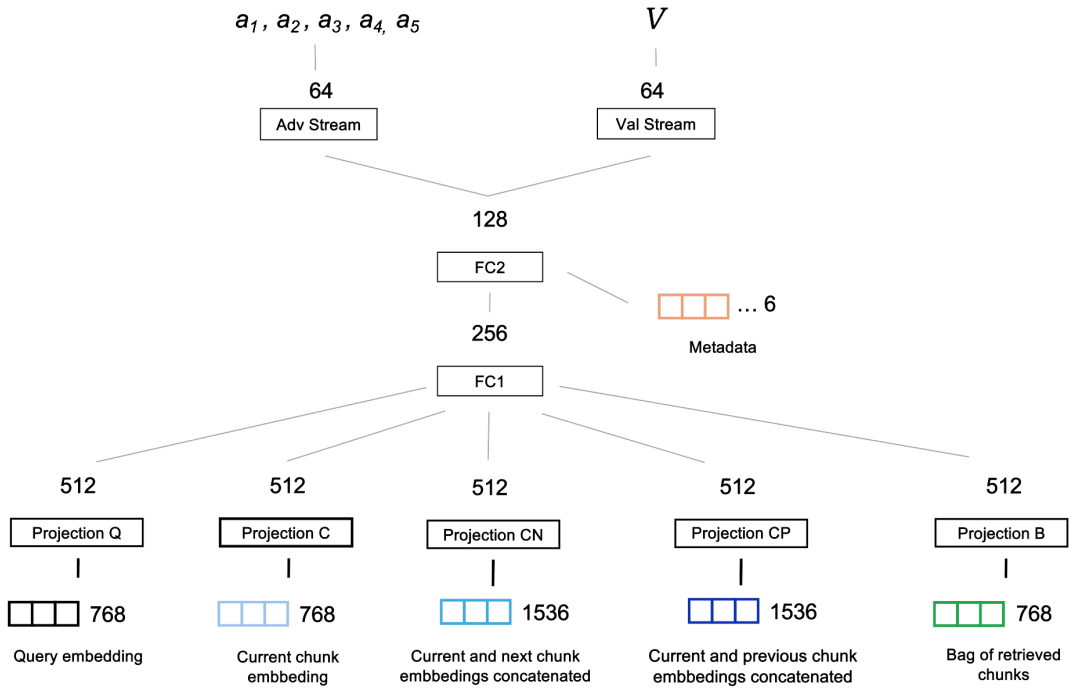
All the text embeddings included in the state representation are going through separate projections. The query, the current (central) chunk and the bag of chunks are projected through 768→512 linear transformations, while the concatenations of the current chunks with its next and previous chunks are projected through the two different 1536→512 transformations, thought to allow the network extract the relations between the central chunk and its adjacent chunks. All projections are followed by ReLU activation.

4.2.2 Hidden Layers and Outputs

As reported in Figure 5.2., the projected embeddings are concatenated into a 1280-dimensional feature vector (5×512), which feeds into a shared trunk consisting of:

- A fully-connected layer mapping to 256 units with ReLU activation, strongly compressing the information flow produced by the projections.

Figure 4.2.
DQN Structure



Note. Dueling Deep Q-Network architecture. In the bottom, we find the embedding features. Every embedding has its own projection. The metadata is injected after the first fully connected layer (FC1). In the top, we find actions outputs and the state value output.

- Concatenation with the metadata vector
- A second fully-connected layer mapping to 128 units with ReLU activation

From this shared representation, the architecture diverges into two streams:

- Value Stream: Estimates the state value function $V(s)$ through a 128→64→1 pathway with ReLU activation in the hidden layer.
- Advantage Stream: Estimates the advantage function $A(s, a)$ for each action through a 128→64→5 pathway, where 5 is the action space dimensionality.

Following the dueling architecture formulation, we combine the value and advantage streams using the mean-subtraction aggregation rule:

$$Q(s,a) = V(s) + A(s,a) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(s,a')$$

This formulation ensures identifiability while allowing the network to learn which states are valuable independent of the action choice, improving learning stability and sample efficiency in environments where many actions share similar values like in this case.

4.3 Resampling and Curriculum Learning

The obtained dataset exhibited a strong class imbalance, with a substantially higher number of irrelevant (negative) chunks compared to relevant (positive) ones. To mitigate this imbalance, an undersampling strategy was applied during training. Specifically, irrelevant chunks were sampled with a controlled probability, thereby increasing the relative proportion of relevant chunks within each retrieval episode.

The degree of undersampling was calibrated empirically using the validation set. Different sampling probabilities were tested, and the final configuration was selected to achieve a convenient trade-off between F1 score and recall.

In addition, a curriculum learning strategy (Bengio et al., 2009) was integrated into the sampling process. Curriculum learning proposes that models benefit from being exposed first to easier training instances and progressively to more difficult ones, improving convergence and generalization. This was implemented by gradually increasing the probability of sampling irrelevant chunks over training epochs, so that in early phases the model had higher chances of retrieving relevant chunks and learning which of the three retrieval options to take, while later in the training it was exposed to a larger amount of irrelevant

chunks to avoid. Specifically, with an initial probability of 70% only relevant chunks were extracted in an episode, and the probability was linearly decreased to 60% until epoch 20, after which remained constant for the rest of the training.

4.4 Prioritized Experience Replay

The implementation incorporates Prioritized Experience Replay (PER) to improve sample efficiency during training (Schaul et al., 2015). Unlike uniform sampling, which treats all experiences equally, PER assigns sampling priorities based on the magnitude of temporal-difference (TD) errors, allowing the agent to learn more frequently from experiences where its predictions are most inaccurate.

4.4.1 Sum Tree Data Structure

The PER implementation utilizes a sum tree data structure for efficient priority-based sampling. The sum tree is a binary tree where each leaf node stores an individual priority value, and each internal node contains the sum of its children's values. This structure enables $O(\log n)$ complexity for both priority updates and sampling operations.

When adding new experiences, priorities are stored at leaf positions, and the sum propagates upward through parent nodes to maintain consistency. Sampling is performed by generating a random value uniformly distributed between 0 and the total priority sum, then traversing the tree from root to leaf to locate the corresponding experience. This approach ensures that the probability of selection is proportional to the assigned priorities.

4.4.2 Priority Assignment and Updates

New experiences are initially assigned the maximum priority observed thus far, ensuring that novel experiences are sampled at least once. The priority p of each experience is computed as:

$$p = (|\delta| + \varepsilon)^{\alpha}$$

where δ represents the TD error, ε is a small constant to prevent zero priorities, and α controls the degree of prioritization. Setting $\alpha = 0$ recovers uniform sampling, while $\alpha = 1$ applies full prioritization proportional to TD error magnitude, in the final model this parameter was set to 0.6.

4.4.3 Importance Sampling Correction

Prioritized sampling introduces bias, as experiences are no longer drawn from the true data distribution. To correct for this bias, importance sampling weights are applied during gradient computation. The weight for experience i is calculated as:

$$w_i = \frac{(N \cdot P(i))^{-\beta}}{\max_j w_j}$$

where N is the current buffer size, $P(i)$ is the sampling probability, and β (set to 0.4) controls the degree of bias correction. The implementation linearly anneals β from its initial value toward 1.0 throughout training, fully compensating for the sampling bias by the end of training.

The weighted loss function is computed as:

$$L = \frac{1}{|B|} \sum_{i \in B} w_i \, \ell(\delta_i)$$

where ℓ denotes the smooth L1 loss function and B represents the sampled batch. This weighting scheme ensures that the gradient updates remain unbiased while still benefiting from the improved sample efficiency of prioritized replay.

4.4.4 Implementation Details

The implementation maintains a circular buffer with fixed capacity, overwriting the oldest experiences when full. Experiences are stored on CPU memory to reduce GPU memory consumption, and transferred to the GPU device only when sampled for training. The buffer stores seven-tuples containing state embeddings, state metadata, actions, rewards, next-state embeddings, next-state metadata, and terminal flags.

4.5 Temporal Difference

At each interaction step, the transition tuple:

(state, action, reward, next state, dones)

was stored in the buffer detailed in the previous section. Gradient updates were performed by sampling mini-batches from this buffer according to the described prioritized replay probabilities.

The online network was optimized by minimizing the smooth L1 loss between predicted Q-values using the online network and TD targets, recursively defined using the target

network as:

$$Q_{\text{target}}(s_t, a_t) = r_t + \gamma\, Q_{\text{target}}\Big(s_{t+1}, \arg\max_a Q_{\text{online}}(s_{t+1}, a)\Big)$$

This is compliant to the double DQN approach, where the target network Q_{target} is used to evaluate the actions that are taken by the online network Q_{online} for the future rewards discounted by factor γ (set to 0.99 in this work).

Through TD minimization, the agent learned a policy mapping states and actions to Q-values representing expected cumulative returns, effectively internalizing both immediate relevance and long-term quality.

4.6 Optimizer and Scheduler

The Adam optimizer (Kingma et al., 2015) was employed for training the DQN network, configured with a learning rate of $\alpha = 2 \cdot 10^{-5}$ and a weight decay (L2 regularization) term set to 10^{-4}. The inclusion of L2 regularization helped constrain the magnitude of the network weights, reducing the risk of overfitting and improving generalization, which is particularly important in reinforcement learning settings where target estimates are inherently noisy due to bootstrapping. To further enhance training stability and convergence, a Cosine Annealing learning rate scheduler (Loshchilov et al., 2017) was used. The last epoch chosen for the final model, based on the overfitting observed and reported in the section 4.8, was precedent to the Tmax parameter set to 90, thus the minimum learning rate was never reached.

4.7 Exploration vs Exploitation

For every epoch of model training, all the queries within the training set were used. Action selection followed an ε-greedy policy. With probability ε, the agent selected a random action to encourage exploration; with probability 1- ε, the action maximizing the current Q-value estimate was selected (exploitation). The exploration rate ε was initialized to 1 and exponentially decayed after each episode, down to about a 60% probability with the 20th epoch. This schedule allowed extensive exploration during early training while progressively favouring policy exploitation. In classical RL this parameter reaches zero during training, gradually going from a learning phase to an evaluation phase, while in this case, given the limited supervised dataset, we interrupt the training at a certain epoch and move to a greedy evaluation ($\varepsilon = 0$), as detailed in the next section.

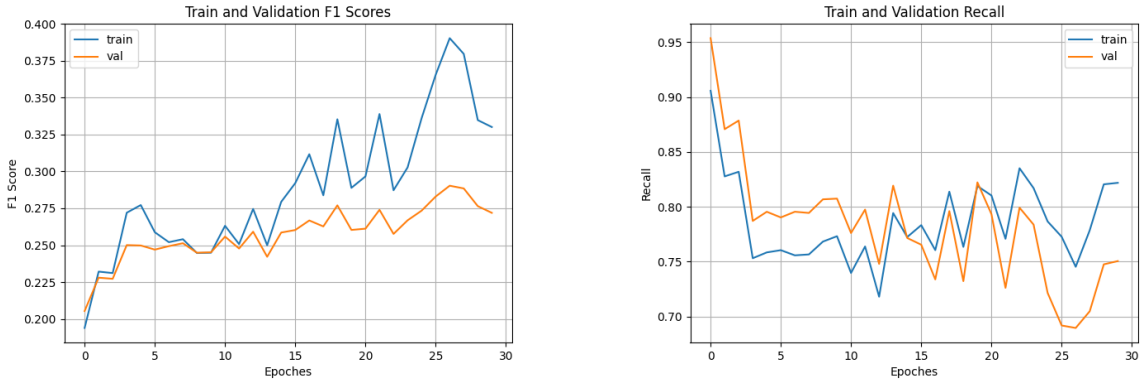
This standard approach was also modified to ensure a complete exploitation of the action space, as during greedy evaluation a steep drop in the usage frequency of some actions was often observed. During exploration, instead of always sampling from a uniform distribution the actions to take, it was taken into account from the train set greedy evaluation which actions were less taken to create a custom distribution aimed at compensating the rarely taken actions. This adapted distribution replaced the uniform distribution whenever the less taken action frequency was below a predefined amount during the training. This approach ensured that the agent used all actions to retrieve or skip chunks and yield to a better performance in all the metrics.

4.8 Greedy Evaluation

To monitor overfitting, after each training epoch the model was evaluated in a greedy inference mode ($\varepsilon = 0$) on both the training and validation sets. During evaluation, the

Figure 4.3. Train and validation scoring across epochs using greedy mode.

(a) F1 score **(b)** Recall



Note. The recall is very high in early epochs because of Curriculum Learning, which is exposing the agent to a limited set of irrelevant chunks.

agent always selected the action with the highest predicted Q-value, without exploration. The resulting average rewards were recorded for each epoch and used to generate comparative learning curves between training and validation performance. As shown in Figure 5.3.a, epoch 21 achieves a promising F1 score of 0.2740 on the validation set, representing an improvement of approximately 6% over the baseline. As desired, this gain is driven by an increase in recall without a decrease in precision. Recall improves by about 10%, rising from 0.6621 to 0.7262, while precision shows a slight decrease from 0.2123 to 0.2112. Later epochs achieve higher F1 scores, but the training score increases further, widening the gap with the validation score and suggesting overfitting. Additionally, these epochs are associated with a sharp decline in recall, as shown in Figure 5.3.b. For this reason, epoch 21 was set as last for the final model training based on the whole training data, to later make the test set final evaluations, discussed in the next chapter.

Chapter 5

Analysis of Results

In this chapter, we report a comprehensive evaluation of the trained RL model using the test set, including feature importance and trajectories analysis.

5.1 Metrics Evaluation

As shown in Table 6.1, on the test set the trained RL model surpasses the threshold-based similarity retrieval by 12.8% on the F1 score and by 11% on recall, while also improving precision (+4.8%). However, the improvement comes with a false positive rate increase (+7.6%), probably deriving from the erroneously retrieved adjacent chunks. These metrics are calculated by averaging the results of single queries, the same approach used to monitor performance during training and fine-tuning. By computing the micro-metrics instead, based on the aggregation of all chunks from the different queries, the RL model improves also the false positive rate (FPR), as can be deduced by the confusion matrix reported in Table 6.2 from which we can calculate a FPR of 25.2% against the 26.5% of the baseline.

By filtering in both the candidate and retrieved sets the chunks with a lower and lower

Table 5.1.
Classification metrics for Cosine Similarity and RL model

Metric	Cosine Similarity	RL model
F1 Score	0.2495	0.2814
Recall	0.6029	0.6696
Precision	0.1963	0.2058
FPR	0.3107	0.3343

Table 5.2.
Confusion matrices for Cosine Similarity and RL model

(a) Cosine Similarity

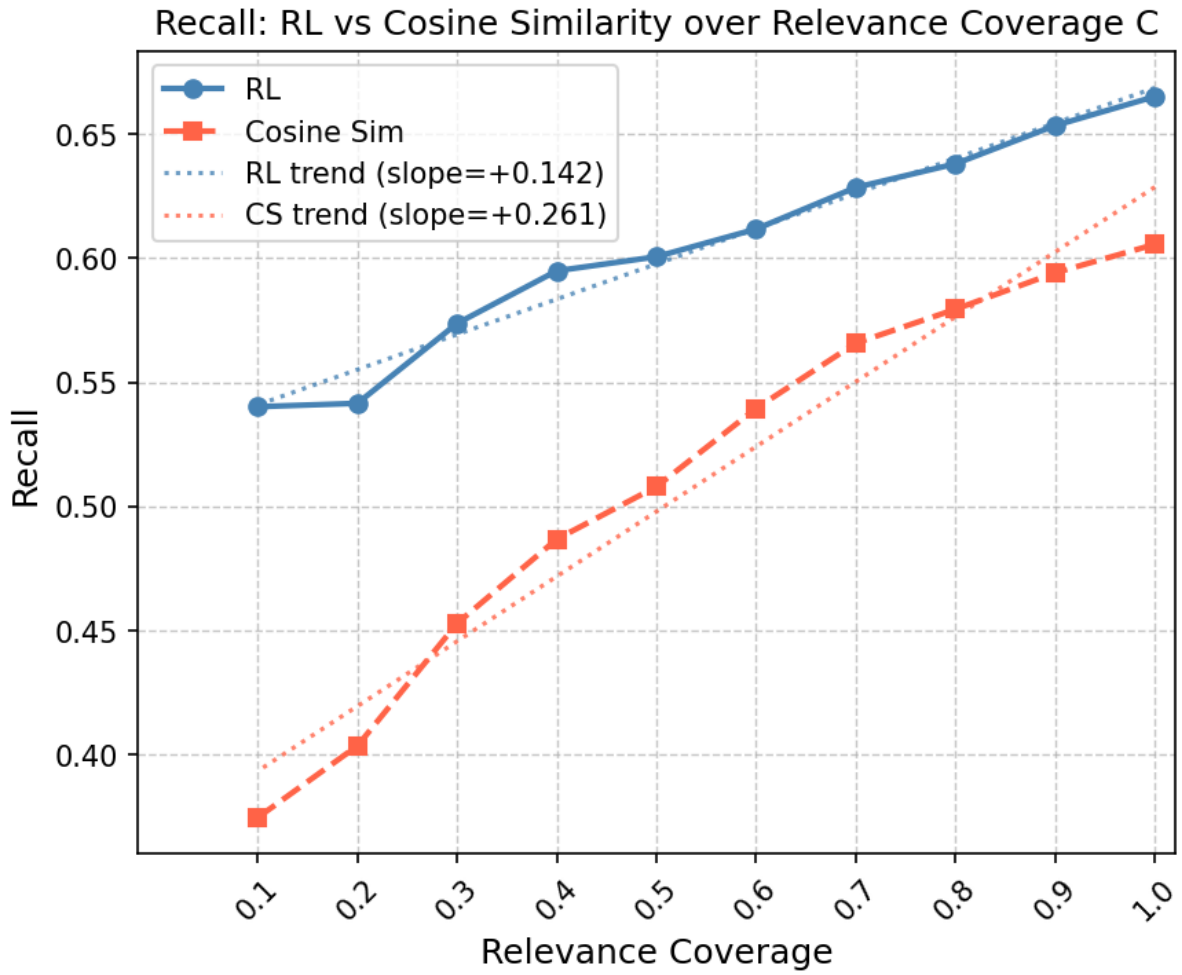
	Pred. Pos	**Pred. Neg**
Actual Pos	2,044	1,805
Actual Neg	12,367	34,222

(b) RL model

	Pred. Pos	**Pred. Neg**
Actual Pos	2,338	1,511
Actual Neg	11,737	34,852

relevance coverage (C), the recall improvement gap gets larger. This is shown in Figure 6.1, by looking at the chart from right to left. This behavior denotes the ability of the developed model to retain relevant chunks whose relevant content is a smaller portion compared to the full chunk size (chunks with relevance coverage $C < 1$).

Figure 5.1.
Recall scores of the Cosine Similarity and of the RL model filtering chunks having an increasing portion from a relevant passage, up to fully relevant chunks.



5.2 Feature Importance

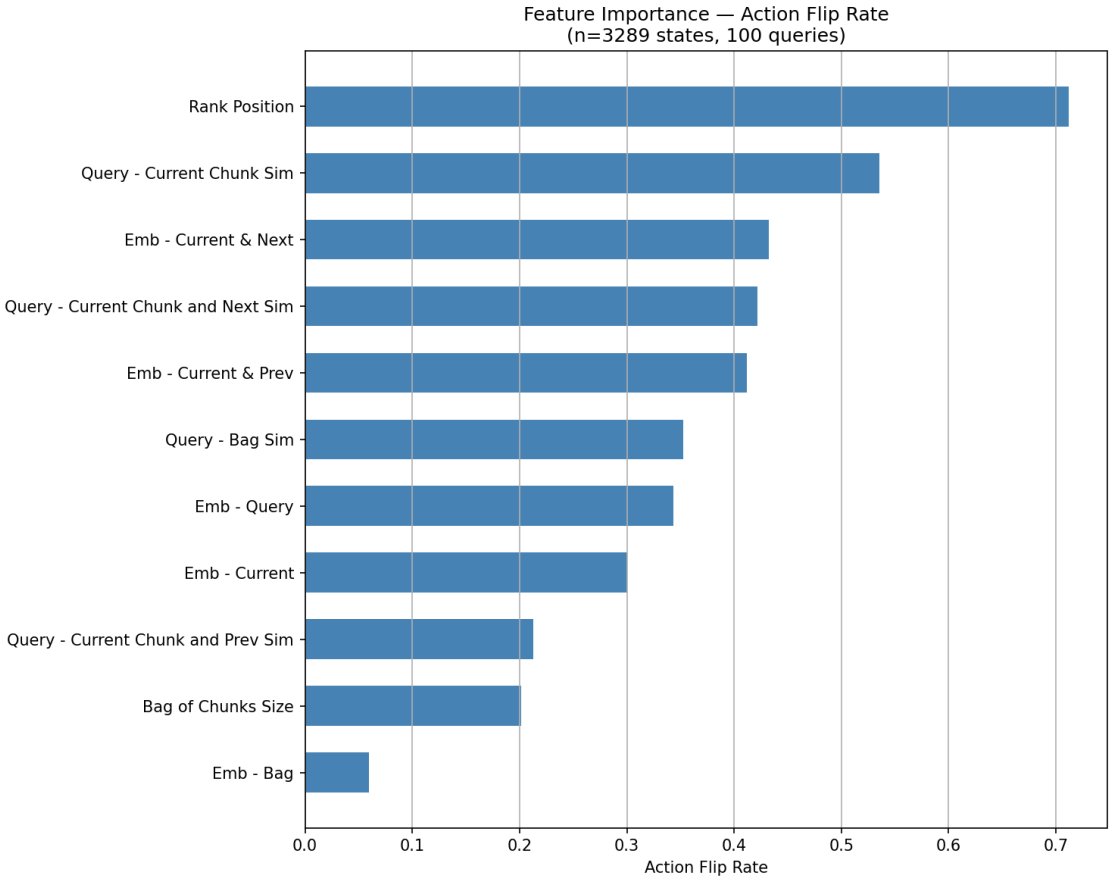
The analysis reported in this section is useful to determine the contribution of the input features in the chunks retrieval of the developed RL model.

5.2.1 Ablation

To assess the relative contribution of both embedding and metadata features, into the model decision making, we use an ablation-based feature importance analysis, inspired by occlusion sensitivity methods (Zeiler et al., 2014). This is implemented by zeroing out individual features and by observing the action flip rate, computed as the fraction of states that brought to a different action compared with the same states from the original model. This was the chosen approach to compare metadata and embeddings as it doesn't depend on the architecture or the gradient paths, which wouldn't allow for a fair comparison given that the metadata is injected in the hidden layers while the raw embeddings belong to the input layer.

By observing the chart, we first note that the model decisions are primary driven by the normalized rank position and by the current chunk similarity with the query. Second importance is given to the embeddings concatenations of the current chunk with the next and previous chunks. Strangely, the next chunk similarity action flip rate is substantially larger than the previous chunk rate. Other features are less used, in particular the bag of chunks embedding. This might be due to the simplicity of its design, indicating that a simple mean of the embedding vectors might not be a faithful representation of the retrieved chunks. Supposedly, this might be the case especially of episodes that are gathering a large number of chunks. However, different results are found in the saliency analysis, regarding only the embedding features, discussed in the next section.

Figure 5.2.
Action flip rate per feature after individual ablation.

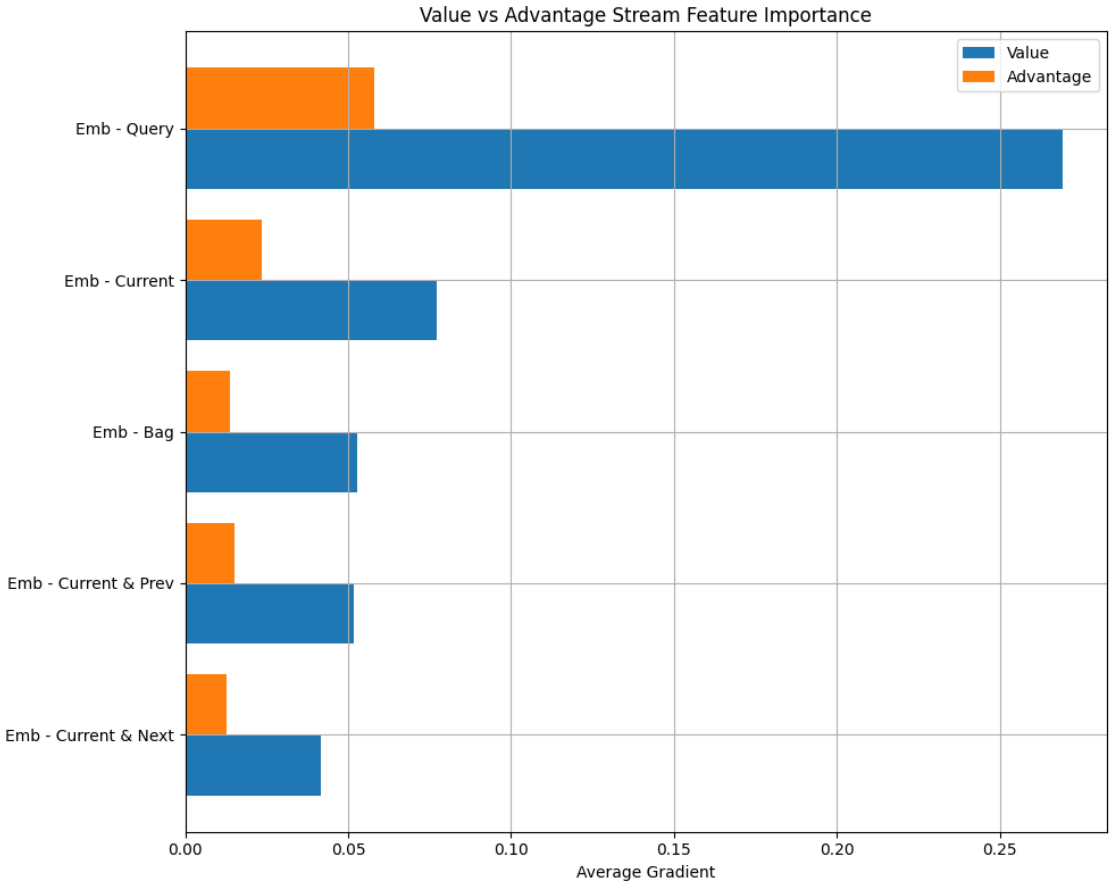


5.2.2 Saliency

As similarly done in the Dueling DQN research work (Wang Z. et al., 2016), we explore the saliency of the model's embedding inputs with respect to the advantage and the value streams separately. For each visited state in a set of sampled episodes, we compute two independent backward passes: one from the scalar value output and one from the advantage outputs. The mean absolute gradient with respect to each input group is then averaged across all visited states, yielding a stream-specific importance score for each embedding item and each metadata feature.

These scores are plotted in a horizontal bar-chart (Figure 6.3.) to compare importance among all embedding features and for each how it's distributed between the two streams on the same scale. By observing the chart, the query clearly presents a major importance,

Figure 5.3.
Saliency analysis of the embedding features.



as it is the main driver to identify the relevant chunks and because we can expect the model to use it in combination with all of the other embedding features. Among the remaining embeddings, it is reasonable to find the current chunk and the bag of embeddings at the second and third positions and the two combinations of the surrounding chunks with the current as last features.

Regarding the advantage and value streams, it appears that the dueling framework is not really exploited by the trained model, since the proportion between the two is consistent among the most used input features, with the value stream much more developed than the advantage stream. However, we find that the bag embedding presents a slightly lower advantage importance, as this metric is a proxy of how good the agent is performing in the episode rather then an information to discriminate between the actions.

Table 5.3.
Action counts and reward statistics for the RL model

Action	Frequency	Avg Reward	Success rate
Skip	8398	-0.0364	92%
Take Curr	6649	-0.0751	13%
Take Curr and Next	761	-0.0993	22%
Take Curr and Prev	303	-0.1577	13%
Take Triple	2065	-0.1150	30%

5.3 Trajectories Analysis

Using a model log, it has been possible to reconstruct the sequence of actions taken for all queries. By aggregating these transitions from the test set inference process, it was possible to extract the statistics reported in table 6.3, reporting action frequencies, their average reward and success rate, computed as the count of non-negative rewarded transitions over the total frequency. The large frequency and success rate of the skip action reflects the dataset distribution, while the reward seems to be too kind on the "Take Current" action considering the low success rate and the action frequency.

To share practical examples, in Appendix A are included examples of chunks from the test set. Example A.1 showcases a fully relevant chunk (21) and its adjacent partially relevant chunks (20 and 21) with the relevant passages regarding the query "Antioxidant / History" underlined. Both of the adjacent chunks, presumably due to the smaller relevant portion, were not retrieved using the cosine similarity method. Instead, the RL model retrieved both the chunks using the "Take Triple" action when examining chunk 21, the first chunk found in the rank. Example A.2 is another successful case, regarding the "Take Current and Next" action. Additionally, we attach in Appendix B an example of a successful trajectory in Table B.1 and an unsuccessful trajectory in table B.2.

Chapter 6

Future Work

This work introduced a reinforcement learning model trained to improve text chunk retrieval by leveraging and extending cosine similarity-based ranking. The system was evaluated on a benchmark derived from the TREC-CAR dataset, demonstrating consistent improvements over the baseline across F1 score, recall, and precision on the test set. The core contribution lies in the agent's ability to recover relevant contextual information that is fragmented by fixed-size chunking, by dynamically deciding whether to expand chunk selections to include surrounding chunks from the original document.

Despite these encouraging results, the ablation analysis reveals a significant limitation: the model's decision-making is predominantly driven by the normalized rank position and cosine similarity scores rather than by the richer semantic information encoded in the chunk embeddings.

Several directions for future research emerge. First, improvements in the representation of retrieved context could allow the model to better capture semantic relationships between chunks. For instance, the current bag-of-chunks representation relies on a simple mean of embedding vectors, which may not accurately reflect the combined semantic meaning of the retrieved passages. More expressive aggregation mechanisms, such as attention-based representations or learned pooling strategies, could provide a more informative summary of the selected content.

Secondly, to push the model toward a more meaningful use of vector representations, a promising approach would involve training dedicated classification models that evaluate

each chunk with its adjacent text portions. Using an ensemble approach, the trained classifiers and the additional metadata could be later used in a similar reinforcement learning framework, sequentially evaluating what to retrieve. This would allow to treat the retrieval problem, concerning a particular topic, with a more comprehensive perspective rather than treating individually every portion of text. Ideally, the RL model could understand what's covered, what's missing and what's redundant to answer the query, and explore the rank or the page to complete the retrieval.

Third, the model is evaluated on a single benchmark dataset, which limits the assessment of its generalization capabilities. Future work should investigate its performance across datasets drawn from different distributions. Additionally, the benchmark relies on relevance labels defined at the chunk level, and a finer-grained supervision, such as sentence-level annotations, could enable more precise evaluation and training signals.

Finally, given the observed ability of the RL model to improve recall and F1 score with minimal increase in false positive rate, integration into a full end-to-end RAG pipeline is a logical extension of this work. Evaluating the downstream impact on answer quality using LLM-based metrics, and the associated computational costs, would provide a more complete picture of the practical value of the retrieval improvements achieved here.

References

Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., ... & McGrew, B. (2023). Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.

Bengio, Y., Louradour, J., Collobert, R., & Weston, J. (2009, June). Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning* (pp. 41-48).

Bommasani, R. (2021). On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*.

Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., & Zaremba, W. (2016). Openai gym. *arXiv preprint arXiv:1606.01540*.

Do Amaral, Vitor Mateus R., et al. "Analysis of the Chunking Process in RAG Architectures." *Escola Regional de Aprendizado de Máquina e Inteligência Artificial da Região Sul (ERAMIA-RS)*. SBC, 2025.

Gomez-Cabello, C. A., Prabha, S., Haider, S. A., Genovese, A., Collaco, B. G., Wood, N. G., Bagaria, S., & Forte, A. J. (2025). Comparative Evaluation of Advanced Chunking for Retrieval-Augmented Generation in Large Language Models for Clinical Decision Support. *Bioengineering (Basel, Switzerland)*, 12(11), 1194.

Iyer, S., Min, S., Mehdad, Y., & Yih, W. T. (2021, June). RECONSIDER: improved reranking using span-focused cross-attention for open domain question answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association*

for Computational Linguistics: Human Language Technologies (pp. 1280-1287).

Ju, M., Yu, W., Zhao, T., Zhang, C., & Ye, Y. (2022). Grape: Knowledge graph enhanced passage reader for open-domain question answering. *arXiv preprint arXiv:2210.02933*.

Karpukhin, V., Oguz, B., Min, S., Lewis, P. S., Wu, L., Edunov, S., ... & Yih, W. T. (2020, November). Dense Passage Retrieval for Open-Domain Question Answering. In *EMNLP (1)* (pp. 6769-6781).

Kingma, D., Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 1412(6).

Kwiatkowski, T., Palomaki, J., Redfield, O., Collins, M., Parikh, A., Alberti, C., ... & Petrov, S. (2019). Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7, 453-466.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33, 9459-9474.

Loshchilov, I., & Hutter, F. (2016). Sgdr: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983*.

Lu, W., Chen, K., Qiao, R., & Sun, X. (2025). Hichunk: Evaluating and enhancing retrieval-augmented generation with hierarchical chunking. *arXiv preprint arXiv:2509.11552*.

Nanni, F., Mitra, B., Magnusson, M., & Dietz, L. (2017, October). Benchmark for complex answer retrieval. In *Proceedings of the ACM SIGIR international conference on theory of information retrieval* (pp. 293-296).

Qu, R., Tu, R., & Bao, F. (2025, April). Is semantic chunking worth the computational cost?. In *Findings of the Association for Computational Linguistics: NAACL 2025* (pp. 2155-2177).

Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., & Manning, C. D. (2024, May). Raptor: Recursive abstractive processing for tree-organized retrieval. In *The Twelfth International Conference on Learning Representations*.

Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2015). Prioritized experience replay. *arXiv preprint arXiv:1511.05952*.

Shuster, K., Poff, S., Chen, M., Kiela, D., & Weston, J. (2021). Retrieval augmentation reduces hallucination in conversation. *arXiv preprint arXiv:2104.07567*.

Van Hasselt, H., Guez, A., & Silver, D. (2016, March). Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence* (Vol. 30, No. 1).

Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, D., ... & Wei, F. (2022). Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*.

Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., & Freitas, N. (2016, June). Dueling network architectures for deep reinforcement learning. In *International conference on machine learning* (pp. 1995-2003). PMLR.

Wu, X., Peng, Z., Sai, K. S. R., Wu, H. T., & Fang, Y. (2024). Passage-specific prompt tuning for passage reranking in question answering with large language models. *arXiv preprint arXiv:2405.20654*.

Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W., Salakhutdinov, R., & Manning, C. D. (2018). HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing* (pp. 2369-2380).

Zeiler, M. D., & Fergus, R. (2014, September). Visualizing and understanding convolutional networks. In *European conference on computer vision* (pp. 818-833). Cham: Springer International Publishing.

Zhang, Z., Zhou, W., Shi, J., & Li, H. (2023). Hybrid and Collaborative Passage Reranking. *arXiv preprint arXiv:2305.09313*.

Appendix A

Examples of Chunks

Example A.1. Triple chunk action recovering both the adjacent relevant chunks for the query "Antioxidant / History". The relevant passage is underlined. Basline only retrieved chunk 21.

- **Chunk 20:** "is eaten and there is high consumption of phytic acid from beans and unleavened whole grain bread. As part of their adaptation from marine life, terrestrial plants began producing non-marine antioxidants such as ascorbic acid (vitamin C), polyphenols and tocopherols. The evolution of angiosperm plants between 50 and 200 million"

- **Chunk 21:** "years ago resulted in the development of many antioxidant pigments 2013 particularly during the Jurassic period 2013 as chemical defences against reactive oxygen species that are byproducts of photosynthesis. Originally, the term antioxidant specifically referred to a chemical that prevented the consumption of oxygen. In the late 19th and early"

- **Chunk 22:** "20th centuries, extensive study concentrated on the use of antioxidants in important industrial processes, such as the prevention of metal corrosion, the vulcanization of rubber, and the polymerization of fuels in the fouling of internal combustion engines. Tirilazad is an antioxidant steroid derivative that inhibits the lipid peroxidation that is"

Example A.2. Successful ”Take Current and Next chunk” action related to the query ”Olive oil / Varieties”. The relevant passage is underlined. Baseline only retrieved chunk 28.

- **Chunk 28:** ”battering the olives. Many ancient presses still exist in the Eastern Mediterranean region, and some dating to the Roman period are still in use today. There are many different olive varieties or olives, each with a particular flavor, texture, and shelf life that make them more or less suitable for”

- **Chunk 29:** ”different applications such as direct human consumption on bread or in salads, indirect consumption in domestic cooking or catering, or industrial uses such as animal feed or engineering applications. The first recorded oil extraction is known from the Hebrew Bible and took place during the Exodus from Egypt, during the”,

Appendix B

Trajectories

Table B.1.
Transitions log from the query "Aquaculture of salmonids / Species / Steelhead"

Chunk ID	Action	Reward
0	Take Triple	0.2955
4	Take Triple	0.8474
74	Take Curr and Next	0.1384
73	Take Curr	0.2702
48	Take Curr and Next	-0.2686
2	Take Curr	0.2671
50	Take Curr	-0.1333
23	Take Curr	-0.1326
24	Take Curr	-0.1319
44	Take Curr	0.2645
82	Take Curr	-0.1317
101	Take Curr	-0.1312
10	Take Curr	-0.1307
84	Take Curr	-0.1303
108	Take Curr	-0.1299
109	Take Curr	-0.1296
63	Skip	0.0000
1	Take Curr	0.2618
98	Take Curr	-0.1295
28	Skip	0.0000
40	Skip	0.0000
95	Take Curr	-0.1293
46	Skip	0.0000
12	Take Curr	-0.1290
30	Skip	0.0000
64	Skip	0.0000
51	Skip	0.0000
29	Skip	0.0000
58	Skip	0.0000
21	Skip	0.0000
83	Skip	0.0000
41	Skip	0.0000
106	Skip	0.0000
9	Skip	0.0000
59	Skip	0.0000
90	Skip	0.0000
45	Skip	-0.4500
20	Skip	0.0000
76	Skip	0.0000

Note. The trajectory of this query yield to a F1 score of 0.53 against the 0.33 from the baseline. Relevant chunks are from 3 relevant passages: chunks from 0 to 5, 73 and 74, 44 and 45.

Table B.2.
Transitions log from the query "Antibiotics / Interactions / Alcohol"

Chunk ID	Action	Reward
81	Take Triple	0.5179
78	Take Curr and Next	-0.2817
11	Take Curr and Next	0.1503
10	Take Curr and Next	0.6428
29	Take Curr	-0.1378
44	Take Curr	-0.1360
8	Take Curr	-0.1345
13	Take Curr	-0.1333
9	Take Curr	-0.1324
30	Take Curr	-0.1315
45	Take Curr	-0.1308
76	Take Curr and Next	-0.2600
68	Skip	0.0000
6	Skip	0.0000
69	Take Curr	-0.1293
38	Take Curr	-0.1290
88	Take Curr	-0.1286
53	Skip	0.0000
15	Take Curr	-0.1283
57	Skip	0.0000
22	Take Curr	-0.1281
14	Take Curr	-0.1278
86	Take Curr	-0.1276
39	Skip	0.0000
17	Take Curr	-0.1275
7	Skip	0.0000
54	Take Curr	-0.1273
0	Skip	0.0000
25	Skip	0.0000
49	Take Curr	-0.1272
90	Skip	0.0000
70	Skip	0.0000
26	Skip	0.0000
89	Skip	0.0000
5	Skip	0.0000
55	Skip	0.0000
36	Skip	0.0000
32	Skip	0.0000
56	Skip	0.0000

Note. The trajectory of this query yield to a F1 score of 0.26 against the 0.5 from the baseline. Relevant chunks are from two passages: 10 and 11, 80 and 81.